

# SQL DDL COMMANDS – COMPLETE CLASS NOTES

## PART 1: INTRODUCTION TO SQL

### 1. What Is SQL?

SQL stands for **Structured Query Language**.

SQL is used to communicate with relational databases.

Using SQL, we can:

- Create databases
  - Create tables
  - Insert data
  - Read data
  - Update data
  - Delete data
  - Modify table structure
  - Create constraints
  - Manage database objects
- 

### 2. What Is a Database?

A database is an organized collection of data.

Example:

Student database may store:

| Student ID | Name | Course | Mobile     |
|------------|------|--------|------------|
| 101        | Ravi | Python | 9876543210 |
| 102        | Sita | SQL    | 9123456780 |

---

### 3. What Is a Table?

A table stores data in rows and columns.

Example:

```
```sql id="u2k51t" students
```

```
| id | name | course | marks |
|---|---|---|---|
| 1 | Ravi | Python | 85 |
| 2 | Sita | SQL | 90 |
```

Here:

- `students` is table name.
- `id`, `name`, `course`, `marks` are columns.
- Each row is one record.

---

#### # 4. Types of SQL Commands

SQL commands are generally divided into:

```
| Category | Full Form | Purpose |
|---|---|---|
| DDL | Data Definition Language | Defines database structure |
| DML | Data Manipulation Language | Manages table data |
| DQL | Data Query Language | Retrieves data |
| DCL | Data Control Language | Controls access |
| TCL | Transaction Control Language | Manages transactions |
```

---

#### # 5. SQL Command Categories

```
| Category | Commands |
|---|---|
| DDL | CREATE, ALTER, DROP, TRUNCATE, RENAME |
| DML | INSERT, UPDATE, DELETE |
| DQL | SELECT |
| DCL | GRANT, REVOKE |
| TCL | COMMIT, ROLLBACK, SAVEPOINT |
```

---

#### # PART 2: DDL INTRODUCTION

#### ## 6. What Is DDL?

DDL stands for **\*\*Data Definition Language\*\***.

DDL commands are used to define, change, and remove database objects.

Database objects include:

- Database
- Table
- View
- Index
- Schema
- Constraints

---

#### # 7. Main DDL Commands

The main DDL commands are:

1. `CREATE`
2. `ALTER`
3. `DROP`
4. `TRUNCATE`
5. `RENAME`

---

#### # 8. DDL Command Purpose

| DDL Command | Purpose                               |
|-------------|---------------------------------------|
| ---         | ---                                   |
| `CREATE`    | Creates a database object             |
| `ALTER`     | Modifies an existing database object  |
| `DROP`      | Deletes a database object permanently |
| `TRUNCATE`  | Deletes all records from a table      |
| `RENAME`    | Renames a database object             |

---

#### # 9. Important Point About DDL

DDL commands affect the structure of the database.

Examples:

- Creating a table
- Adding a column
- Changing data type

- Removing a table
- Renaming a table

---

## # 10. DDL vs DML

|                               |                                  |
|-------------------------------|----------------------------------|
| DDL                           | DML                              |
| ---                           | ---                              |
| Data Definition Language      | Data Manipulation Language       |
| Works on structure            | Works on data                    |
| Commands: CREATE, ALTER, DROP | Commands: INSERT, UPDATE, DELETE |
| Used to define tables         | Used to manage records           |
| Usually affects schema        | Affects table rows               |

---

## # PART 3: DATA TYPES IN SQL

### ## 11. What Is a Data Type?

A data type defines what kind of data a column can store.

Example:

```
```sql id="ef814s"
name VARCHAR(50)
age INT
salary DECIMAL(10,2)
joining_date DATE
```

## 12. Common SQL Data Types

| Data Type     | Purpose                    | Example      |
|---------------|----------------------------|--------------|
| INT           | Whole numbers              | 10, 100, 500 |
| BIGINT        | Large whole numbers        | 9876543210   |
| DECIMAL(p, s) | Exact decimal values       | 9999.99      |
| FLOAT         | Approximate decimal values | 75.5         |
| CHAR(n)       | Fixed-length text          | 'M', 'YES'   |
| VARCHAR(n)    | Variable-length text       | 'Ravi'       |

| Data Type | Purpose       | Example             |
|-----------|---------------|---------------------|
| TEXT      | Long text     | Description         |
| DATE      | Date values   | 2026-07-15          |
| TIME      | Time values   | 10:30:00            |
| DATETIME  | Date and time | 2026-07-15 10:30:00 |
| BOOLEAN   | True/False    | TRUE, FALSE         |

## 13. INT Data Type

Used to store whole numbers.

```
```sql id="gq6gr9" age INT
```

Example values:

```
```text id="c2uqrw"
18
25
100
```

## 14. VARCHAR Data Type

Used to store variable-length text.

```
```sql id="jqxaj2" name VARCHAR(100)
```

Example values:

```
```text id="ppjd06"
Ravi
Sita
Anurag IT Solutions
```

## 15. CHAR vs VARCHAR

| CHAR                    | VARCHAR                    |
|-------------------------|----------------------------|
| Fixed length            | Variable length            |
| May use extra space     | Uses needed space          |
| Good for fixed data     | Good for names, emails     |
| Example: gender CHAR(1) | Example: name VARCHAR(100) |

## 16. DECIMAL Data Type

Used to store exact decimal numbers.

```
```sql id="4c7pr7" fee DECIMAL(10,2)
```

This means:

- Total digits: 10
- Digits after decimal: 2

Example:

```
```text id="qq8eth"
9999.00
25000.50
```

## 17. DATE Data Type

Used to store date.

```
```sql id="7xtzb7" joining_date DATE
```

Example:

```
```text id="8imp6f"
2026-07-15
```

## 18. DATETIME Data Type

Used to store date and time.

```
```sql id="xx48e9" created_at DATETIME
```

Example:

```
```text id="jscg4j"  
2026-07-15 11:30:00
```

---

## PART 4: CONSTRAINTS IN DDL

### 19. What Is a Constraint?

A constraint is a rule applied to a column or table.

Constraints help maintain:

- Accuracy
- Validity
- Uniqueness
- Relationship
- Data integrity

---

## 20. Common SQL Constraints

| Constraint  | Purpose                             |
|-------------|-------------------------------------|
| NOT NULL    | Column cannot be empty              |
| UNIQUE      | Values must be unique               |
| PRIMARY KEY | Unique identifier for each row      |
| FOREIGN KEY | Creates relationship between tables |
| CHECK       | Validates condition                 |
| DEFAULT     | Provides default value              |

## 21. NOT NULL Constraint

`NOT NULL` means a column must contain a value.

Example:

```
```sql id="flnnc2" name VARCHAR(100) NOT NULL
```

This means `name` cannot be empty.

---

# 22. UNIQUE Constraint

`UNIQUE` means duplicate values are not allowed.

Example:

```
```sql id="ntuxiq"
email VARCHAR(100) UNIQUE
```

No two students can have the same email.

---

## 23. PRIMARY KEY Constraint

A primary key uniquely identifies each row.

Example:

```
```sql id="7fj4qd" id INT PRIMARY KEY
```

Properties:

- Unique
- Not null
- One table usually has one primary key

---

# 24. FOREIGN KEY Constraint

A foreign key creates a relationship between two tables.

Example:

```
```sql id="2fkd29"
course_id INT,
FOREIGN KEY (course_id) REFERENCES courses(id)
```

This connects student table with course table.

---

## 25. CHECK Constraint

**CHECK** validates data using a condition.

Example:

```
```sql id="j04of3" marks INT CHECK (marks >= 0 AND marks <= 100)
```

Marks must be between 0 and 100.

---

# 26. DEFAULT Constraint

`DEFAULT` gives a default value if no value is provided.

Example:

```
```sql id="xbv3wm"
status VARCHAR(20) DEFAULT 'Active'
```

If status is not entered, it becomes **Active**.

---

## PART 5: CREATE COMMAND

### 27. What Is CREATE Command?

The **CREATE** command is used to create database objects.

We can create:

- Database
  - Table
  - View
  - Index
  - Schema
- 

## 28. Create Database

Syntax:

```
```sql id="zubn73" CREATE DATABASE database_name;
```

Example:

```
```sql id="bs49g9"
CREATE DATABASE training_db;
```

---

## 29. Use Database

After creating a database, select it:

```
```sql id="267sie" USE training_db;
```

```
---
```

```
# 30. Create Table Syntax
```

```
```sql id="d8furp"
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    column3 datatype constraint
);
```

---

## 31. Create Simple Student Table

```
```sql id="l4c6bb" CREATE TABLE students ( id INT, name VARCHAR(100), course VARCHAR(100), marks INT );
```

```
---  
  
# 32. Create Table with Primary Key  
  
```sql id="062swr"  
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    course VARCHAR(100),  
    marks INT  
);
```

---

## 33. Create Table with NOT NULL

```
```sql id="zmkd6w" CREATE TABLE students ( id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, course VARCHAR(100) NOT NULL, marks INT );
```

```
---  
  
# 34. Create Table with UNIQUE  
  
```sql id="bpblqn"  
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    course VARCHAR(100)  
);
```

---

## 35. Create Table with CHECK

```
```sql id="j8g89y" CREATE TABLE students ( id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, marks INT CHECK (marks >= 0 AND marks <= 100) );
```

```

---

# 36. Create Table with DEFAULT

```sql id="8qz8mo"
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    course VARCHAR(100),
    status VARCHAR(20) DEFAULT 'Active'
);

```

## 37. Create Table with Multiple Constraints

```

```sql id="60e3tk" CREATE TABLE students ( id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, email
VARCHAR(100) UNIQUE, age INT CHECK (age >= 18), course VARCHAR(100) DEFAULT 'Python', status
VARCHAR(20) DEFAULT 'Active' );

```

```

---

# 38. Create Courses Table

```sql id="1e3x2a"
CREATE TABLE courses (
    id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    duration VARCHAR(50),
    fee DECIMAL(10,2)
);

```

## 39. Create Students Table with Foreign Key

```

```sql id="rl6k rn" CREATE TABLE students ( id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, course_id
INT, FOREIGN KEY (course_id) REFERENCES courses(id) );

```

```

---

# 40. Create Table with Auto Increment - MySQL

```

```
```sql id="v3gvqk"
CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    course VARCHAR(100)
);
```

---

## 41. Create Table with Auto Increment – SQL Server

```
```sql id="caet7h" CREATE TABLE students ( id INT PRIMARY KEY IDENTITY(1,1), name VARCHAR(100) NOT
NULL, email VARCHAR(100) UNIQUE, course VARCHAR(100) );
```

```
---

# 42. Create Table with Auto Increment – SQLite

```sql id="9jwsat"
CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    course TEXT
);
```

---

## 43. Create Employee Table

```
```sql id="qid0ex" CREATE TABLE employees ( emp_id INT PRIMARY KEY, emp_name VARCHAR(100) NOT
NULL, department VARCHAR(50), salary DECIMAL(10,2), joining_date DATE, status VARCHAR(20) DEFAULT
'Active' );
```

```
---

# 44. Create Product Table

```sql id="802vf6"
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) CHECK (price > 0),
```

```
stock INT DEFAULT 0
);
```

---

## 45. Create Customer Table

```
```sql id="e663oe" CREATE TABLE customers ( customer_id INT PRIMARY KEY, customer_name VARCHAR(100)
NOT NULL, mobile VARCHAR(15) UNIQUE, email VARCHAR(100) UNIQUE, city VARCHAR(50) );
```

```
---

# 46. Create Orders Table

```sql id="tnyige"
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

---

## PART 6: ALTER COMMAND

### 47. What Is ALTER Command?

The `ALTER` command is used to modify an existing table structure.

Using `ALTER`, we can:

- Add a column
- Modify column data type
- Rename a column
- Drop a column
- Add constraints
- Drop constraints
- Rename table

## 48. Add New Column

Syntax:

```
```sql id="u646gc" ALTER TABLE table_name ADD column_name datatype;
```

Example:

```
```sql id="cs80ae"
ALTER TABLE students
ADD mobile VARCHAR(15);
```

---

## 49. Add Multiple Columns – MySQL Example

```
```sql id="2g86ws" ALTER TABLE students ADD address VARCHAR(255), ADD city VARCHAR(50);
```

Some databases may use slightly different syntax.

---

# 50. Add Column with Constraint

```
```sql id="py2mcv"
ALTER TABLE students
ADD email VARCHAR(100) UNIQUE;
```

---

## 51. Add Column with Default Value

```
```sql id="7pvqj6" ALTER TABLE students ADD status VARCHAR(20) DEFAULT 'Active';
```

---

# 52. Modify Column Data Type – MySQL

```
```sql id="sgne6f"
ALTER TABLE students
MODIFY name VARCHAR(150);
```

---

## 53. Modify Column Data Type – SQL Server

```
```sql id="52h1wy" ALTER TABLE students ALTER COLUMN name VARCHAR(150);
```

```
---  
  
# 54. Change Column Name - MySQL  
  
```sql id="jtbk0u"  
ALTER TABLE students  
CHANGE mobile phone_number VARCHAR(15);
```

---

## 55. Rename Column – MySQL 8+

```
```sql id="8iw6bm" ALTER TABLE students RENAME COLUMN mobile TO phone_number;
```

```
---  
  
# 56. Rename Column - SQL Server  
  
```sql id="u7e6mj"  
EXEC sp_rename 'students.mobile', 'phone_number', 'COLUMN';
```

---

## 57. Rename Column – PostgreSQL / SQLite

```
```sql id="359mde" ALTER TABLE students RENAME COLUMN mobile TO phone_number;
```

```
---  
  
# 58. Drop Column  
  
```sql id="m6j3ls"  
ALTER TABLE students  
DROP COLUMN address;
```

## 59. Add Primary Key Constraint

```
```sql id="1aicxx" ALTER TABLE students ADD PRIMARY KEY (id);
```

```
---
```

```
# 60. Add Unique Constraint
```

```
```sql id="u5iqa3"
```

```
ALTER TABLE students
```

```
ADD CONSTRAINT unique_email UNIQUE (email);
```

---

## 61. Add Check Constraint

```
```sql id="oump3p" ALTER TABLE students ADD CONSTRAINT check_marks CHECK (marks >= 0 AND marks <= 100);
```

```
---
```

```
# 62. Add Foreign Key Constraint
```

```
```sql id="qjjm1g"
```

```
ALTER TABLE students
```

```
ADD CONSTRAINT fk_course
```

```
FOREIGN KEY (course_id)
```

```
REFERENCES courses(id);
```

---

## 63. Drop Constraint – MySQL Primary Key

```
```sql id="1j7nzp" ALTER TABLE students DROP PRIMARY KEY;
```

```
---
```

```
# 64. Drop Foreign Key – MySQL
```

```
```sql id="znzn72"
```

```
ALTER TABLE students
DROP FOREIGN KEY fk_course;
```

## 65. Drop Constraint – SQL Server / PostgreSQL

```
```sql id="acsdnc" ALTER TABLE students DROP CONSTRAINT constraint_name;
```

```
---

# 66. Rename Table Using ALTER

Some databases allow:

```sql id="ny3utn"
ALTER TABLE students
RENAME TO student_details;
```

## 67. ALTER Command Examples Summary

| Task            | SQL Example                                                                  |
|-----------------|------------------------------------------------------------------------------|
| Add column      | <code>ALTER TABLE students ADD mobile VARCHAR(15);</code>                    |
| Drop column     | <code>ALTER TABLE students DROP COLUMN mobile;</code>                        |
| Modify datatype | <code>ALTER TABLE students MODIFY name VARCHAR(150);</code>                  |
| Rename column   | <code>ALTER TABLE students RENAME COLUMN mobile TO phone;</code>             |
| Add constraint  | <code>ALTER TABLE students ADD CONSTRAINT unique_email UNIQUE(email);</code> |

## PART 7: DROP COMMAND

### 68. What Is DROP Command?

The `DROP` command is used to permanently remove database objects.

It can remove:

- Database
  - Table
  - View
  - Index
- 

## 69. Drop Table

Syntax:

```
```sql id="qxp8f8" DROP TABLE table_name;
```

Example:

```
```sql id="n4d9af"  
DROP TABLE students;
```

This permanently deletes:

- Table structure
  - All records
  - Constraints related to the table
- 

## 70. Drop Table If Exists

```
```sql id="tofet6" DROP TABLE IF EXISTS students;
```

This avoids error if table does not exist.

---

# 71. Drop Database

```
```sql id="3x2j1m"  
DROP DATABASE training_db;
```

This permanently deletes the database.

---

## 72. Drop Database If Exists

```
```sql id="c1ttfi" DROP DATABASE IF EXISTS training_db;
```

```
---

# 73. DROP Warning

Use `DROP` carefully.

After dropping a table:

```text id="rdqsof"
Table structure is removed.
All data is removed.
Recovery may not be possible without backup.
```

## 74. DROP vs DELETE

| DROP                                | DELETE                                      |
|-------------------------------------|---------------------------------------------|
| DDL command                         | DML command                                 |
| Removes table structure             | Removes rows                                |
| Deletes table object                | Table remains                               |
| Usually irreversible without backup | Can be transaction-controlled in many cases |

## PART 8: TRUNCATE COMMAND

### 75. What Is TRUNCATE Command?

**TRUNCATE** removes all rows from a table.

It keeps the table structure.

Syntax:

```
```sql id="93I9ye" TRUNCATE TABLE table_name;
```

Example:

```
```sql id="airodw"
TRUNCATE TABLE students;
```

## 76. What Happens After TRUNCATE?

After truncate:

- All rows are removed.
- Table structure remains.
- Columns remain.
- Constraints remain.
- Table can be reused.

## 77. TRUNCATE vs DELETE

| TRUNCATE                                                | DELETE                                        |
|---------------------------------------------------------|-----------------------------------------------|
| DDL command                                             | DML command                                   |
| Removes all rows                                        | Removes selected rows or all rows             |
| Usually faster                                          | Usually slower for large data                 |
| Cannot use WHERE                                        | Can use WHERE                                 |
| Table structure remains                                 | Table structure remains                       |
| May reset identity/auto-increment depending on database | Usually does not reset identity automatically |

## 78. TRUNCATE vs DROP

| TRUNCATE                         | DROP                    |
|----------------------------------|-------------------------|
| Removes all records              | Removes complete table  |
| Table structure remains          | Table structure removed |
| Can insert again into same table | Need to recreate table  |

TRUNCATE

DROP

DDL command

DDL command

## 79. Important TRUNCATE Note

Some databases restrict truncating tables referenced by foreign keys.

Example:

```text id="jt6irc" Cannot truncate parent table if child table depends on it.

Behavior depends on database system.

---

# PART 9: RENAME COMMAND

## 80. What Is RENAME Command?

The `RENAME` command is used to rename a database object.

We can rename:

- Table
- Column
- Database object

---

# 81. Rename Table - General

```
```sql id="ocx3r5"
RENAME TABLE old_table_name TO new_table_name;
```

Example:

```sql id="yw5avj" RENAME TABLE students TO student\_details;

This syntax is common in MySQL.

---

```
# 82. Rename Table - PostgreSQL / SQLite
```

```
```sql id="x7l7gz"
ALTER TABLE students
RENAME TO student_details;
```

---

## 83. Rename Column

```
```sql id="64y9ad" ALTER TABLE students RENAME COLUMN mobile TO phone_number;
```

Supported syntax may vary by database.

---

```
# PART 10: CREATE INDEX
```

```
## 84. What Is an Index?
```

An index improves search performance on table columns.

Example:

If students are frequently searched by email:

```
```sql id="s8bzb8"
CREATE INDEX idx_students_email
ON students(email);
```

---

## 85. Create Index Syntax

```
```sql id="5ao6be" CREATE INDEX index_name ON table_name(column_name);
```

Example:

```
```sql id="rm2qa1"
CREATE INDEX idx_student_course
ON students(course);
```

## 86. Create Unique Index

```
```sql id="m64yj7" CREATE UNIQUE INDEX idx_unique_email ON students(email);
```

This also prevents duplicate email values.

---

# 87. Drop Index - MySQL

```
```sql id="4tpty1f"
DROP INDEX idx_student_course
ON students;
```

---

## 88. Drop Index – PostgreSQL / SQL Server

```
```sql id="rtu78r" DROP INDEX idx_student_course;
```

---

# 89. Index Advantages and Disadvantages

|                         |                                         |
|-------------------------|-----------------------------------------|
| Advantages              | Disadvantages                           |
| ---                     | ---                                     |
| Faster searching        | Uses extra storage                      |
| Faster filtering        | Slower insert/update/delete sometimes   |
| Useful for large tables | Too many indexes can reduce performance |

---

# PART 11: CREATE VIEW

## 90. What Is a View?

A view is a virtual table based on a SQL query.

It does not usually store data separately like a normal table.

Example:

```
```sql id="8s2g6c"
CREATE VIEW active_students AS
```

```
SELECT id, name, course
FROM students
WHERE status = 'Active';
```

---

## 91. Create View Syntax

```
```sql id="qlhwhf" CREATE VIEW view_name AS SELECT columns FROM table_name WHERE condition;
```

```
---

# 92. Create Student View

```sql id="wwpq03"
CREATE VIEW python_students AS
SELECT id, name, email
FROM students
WHERE course = 'Python';
```

---

## 93. Select from View

```
```sql id="q4tnt5" SELECT * FROM python_students;
```

```
---

# 94. Drop View

```sql id="0tohce"
DROP VIEW python_students;
```

---

## 95. Advantages of View

Views are useful for:

- Simplifying complex queries
- Improving security by hiding columns
- Reusing common query logic
- Creating filtered virtual tables

## PART 12: COMPLETE DDL EXAMPLE PROJECT

### 96. Training Institute Database Design

We will create:

1. courses
  2. students
  3. trainers
  4. batches
  5. enrollments
- 

### 97. Create Database

```
```sql id="hlhIk8" CREATE DATABASE institute_db;
```

```
USE institute_db;
```

```
---  
  
# 98. Courses Table  
  
```sql id="lrq4i2"  
CREATE TABLE courses (  
    course_id INT PRIMARY KEY AUTO_INCREMENT,  
    course_name VARCHAR(100) NOT NULL UNIQUE,  
    duration VARCHAR(50),  
    fee DECIMAL(10,2) CHECK (fee >= 0),  
    status VARCHAR(20) DEFAULT 'Active'  
);
```

---

### 99. Students Table

```
```sql id="huf87i" CREATE TABLE students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_name VARCHAR(100) NOT NULL,  
    mobile VARCHAR(15) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    qualification VARCHAR(100),  
    city VARCHAR(50),  
    status VARCHAR(20) DEFAULT 'Active';
```

```

---

# 100. Trainers Table

```sql id="2ht85z"
CREATE TABLE trainers (
    trainer_id INT PRIMARY KEY AUTO_INCREMENT,
    trainer_name VARCHAR(100) NOT NULL,
    technology VARCHAR(100),
    experience_years INT CHECK (experience_years >= 0),
    mobile VARCHAR(15) UNIQUE,
    status VARCHAR(20) DEFAULT 'Active'
);

```

## 101. Batches Table

```

```sql id="oxgd6g" CREATE TABLE batches ( batch_id INT PRIMARY KEY AUTO_INCREMENT, course_id INT,
trainer_id INT, batch_time VARCHAR(50), start_date DATE, mode VARCHAR(20) DEFAULT 'Offline', FOREIGN
KEY (course_id) REFERENCES courses(course_id), FOREIGN KEY (trainer_id) REFERENCES trainers(trainer_id) );

```

```

---

# 102. Enrollments Table

```sql id="9qgjx5"
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    fee_paid DECIMAL(10,2) DEFAULT 0,
    payment_status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

```

## 103. Add Column Later

```

```sql id="p8xiij" ALTER TABLE students ADD parent_mobile VARCHAR(15);

```

```
---

# 104. Modify Column

```sql id="mu99hh"
ALTER TABLE students
MODIFY student_name VARCHAR(150) NOT NULL;
```

---

## 105. Rename Column

```
```sql id="u75p56" ALTER TABLE students RENAME COLUMN parent_mobile TO guardian_mobile;
```

```
---

# 106. Add Index

```sql id="bshjgg"
CREATE INDEX idx_student_mobile
ON students(mobile);
```

---

## 107. Create View

```
```sql id="6y3q47" CREATE VIEW active_enrollments AS SELECT students.student_name,
courses.course_name, enrollments.enrollment_date, enrollments.payment_status FROM enrollments JOIN
students ON enrollments.student_id = students.student_id JOIN courses ON enrollments.course_id =
courses.course_id WHERE students.status = 'Active';
```

```
---

# PART 13: DDL COMMAND DIFFERENCES

## 108. CREATE vs ALTER

| CREATE | ALTER |
|---|---|
| Creates new object | Modifies existing object |
| Used before object exists | Used after object exists |
| Example: create table | Example: add column |
```

---

#### # 109. DROP vs TRUNCATE

| DROP                     | TRUNCATE                     |
|--------------------------|------------------------------|
| Removes table completely | Removes all rows only        |
| Table structure deleted  | Table structure remains      |
| Need to recreate table   | Can insert again immediately |
| DDL                      | DDL                          |

---

#### # 110. DELETE vs TRUNCATE vs DROP

| Command  | Category | Removes           | WHERE Allowed | Structure Remains |
|----------|----------|-------------------|---------------|-------------------|
| DELETE   | DML      | Selected/all rows | Yes           | Yes               |
| TRUNCATE | DDL      | All rows          | No            | Yes               |
| DROP     | DDL      | Entire table      | No            | No                |

---

#### # 111. PRIMARY KEY vs UNIQUE

| PRIMARY KEY                       | UNIQUE                                    |
|-----------------------------------|-------------------------------------------|
| Uniquely identifies each row      | Prevents duplicate values                 |
| Does not allow NULL               | May allow NULL depending on database      |
| Usually one primary key per table | Multiple unique constraints possible      |
| Used as main identifier           | Used for candidate keys like email/mobile |

---

#### # 112. PRIMARY KEY vs FOREIGN KEY

| PRIMARY KEY                   | FOREIGN KEY                             |
|-------------------------------|-----------------------------------------|
| Identifies row in same table  | References primary key in another table |
| Unique and not null           | Can contain repeated values             |
| Parent table key              | Child table reference                   |
| Example: course_id in courses | course_id in students                   |

---

#### # 113. CHAR vs VARCHAR

| CHAR | VARCHAR |
|------|---------|
|------|---------|

```

|---|---|
| Fixed length | Variable length |
| Uses fixed storage | Uses required storage |
| Good for codes | Good for names/emails |
| Example: gender CHAR(1) | name VARCHAR(100) |

---

# 114. CHECK vs DEFAULT

| CHECK | DEFAULT |
|---|---|
| Validates allowed values | Provides automatic value |
| Prevents invalid data | Fills missing value |
| Example: age >= 18 | status default Active |

---

# PART 14: COMMON ERRORS IN DDL

## 115. Table Already Exists

Error occurs when creating a table that already exists.

Solution:

```sql id="t1yln2"
CREATE TABLE IF NOT EXISTS students (
    id INT PRIMARY KEY,
    name VARCHAR(100)
);

```

## 116. Database Already Exists

Solution:

```
```sql id="j3doz8" CREATE DATABASE IF NOT EXISTS training_db;
```

```

---

# 117. Duplicate Column Name

Example:

```

```
```sql id="16tdm1"
ALTER TABLE students
ADD name VARCHAR(100);
```

If `name` already exists, it causes an error.

---

## 118. Foreign Key Error

A foreign key must reference an existing table and compatible column.

Example issue:

```text id="lo747c" students.course\_id references courses.id but courses table does not exist.

Create parent table first.

---

# 119. Data Type Mismatch

Foreign key and referenced primary key should use compatible data types.

Example:

```
```text id="oey1ck"
course_id INT
references
courses.course_id VARCHAR(10)
```

This is not proper design.

---

## 120. Dropping Parent Table

If another table depends on a parent table through foreign key, dropping it may fail.

Example:

```text id="8zgja" courses table is referenced by students table.

Drop child table first or remove foreign key constraint.

---

# PART 15: BEST PRACTICES FOR DDL

## 121. Naming Best Practices

Use meaningful names:

Good:

```
```sql id="kzokqa"
students
student_id
course_name
enrollment_date
```

Avoid:

```
```sql id="ufh2zp" s sid cn ed
```

---

# 122. Table Naming

Recommended:

```
```text id="dyc9fq"
students
courses
employees
orders
products
```

Use lowercase and plural names consistently.

---

## 123. Column Naming

Recommended:

```
```text id="j0b85a" student_id student_name course_id created_at updated_at
```

Use snake\_case for readability.

---

# 124. Always Use Primary Key

Every important table should have a primary key.

Example:

```
```sql id="k7a7se"
student_id INT PRIMARY KEY AUTO_INCREMENT
```

---

## 125. Use NOT NULL for Required Fields

Example:

```
```sql id="w45816" student_name VARCHAR(100) NOT NULL
```

---

# 126. Use UNIQUE for Email and Mobile

Example:

```
```sql id="63835v"
email VARCHAR(100) UNIQUE
mobile VARCHAR(15) UNIQUE
```

---

## 127. Use DEFAULT for Status Fields

Example:

```
```sql id="i275f8" status VARCHAR(20) DEFAULT 'Active'
```

---

# 128. Use CHECK for Valid Ranges

Example:

```
```sql id="am49gw"
marks INT CHECK (marks >= 0 AND marks <= 100)
```

## 129. Create Parent Tables First

When using foreign keys:

```text id="6y0jnz" Create courses table first. Then create students table.

```
---

# 130. Use Consistent Data Types

If parent key is:

```sql id="eguvwe"
course_id INT
```

child foreign key should also be:

```sql id="u3634p" course\_id INT

```
---

# 131. Avoid Unnecessary Columns

Do not create columns that are never used.

Good database design should be:

- Clean
- Simple
- Scalable
- Easy to understand

---

# 132. Keep Backup Before DROP

Before using:
```

```
```sql id="382f1q"  
DROP TABLE students;
```

make sure you have backup if data is important.

---

## 133. Do Not Use Reserved Words

Avoid table or column names like:

```
```text id="9o0iax" order select table user
```

Better:

```
```text id="515612"  
orders  
student_users  
course_table
```

---

## PART 16: DDL INTERVIEW QUESTIONS AND ANSWERS

### 134. What Is DDL?

DDL stands for Data Definition Language.

It is used to create, modify, and delete database objects.

---

### 135. What Are the Main DDL Commands?

Main DDL commands are:

- CREATE
- ALTER
- DROP
- TRUNCATE
- RENAME

---

### 136. What Is CREATE Command?

`CREATE` is used to create database objects such as databases and tables.

---

### 137. What Is ALTER Command?

`ALTER` modifies the structure of an existing database object.

---

### 138. What Is DROP Command?

`DROP` permanently removes a database object.

---

### 139. What Is TRUNCATE Command?

`TRUNCATE` removes all records from a table while keeping the structure.

---

### 140. What Is RENAME Command?

`RENAME` changes the name of a database object.

---

### 141. Difference Between DELETE and TRUNCATE?

`DELETE` is DML and can remove selected rows using `WHERE`.

`TRUNCATE` is DDL and removes all rows without `WHERE`.

---

### 142. Difference Between DROP and TRUNCATE?

`DROP` removes the complete table.

`TRUNCATE` removes only rows and keeps table structure.

---

### **143. What Is a Primary Key?**

A primary key uniquely identifies each row in a table.

---

### **144. What Is a Foreign Key?**

A foreign key creates a relationship between two tables.

---

### **145. What Is NOT NULL?**

It ensures a column cannot store NULL values.

---

### **146. What Is UNIQUE?**

It ensures duplicate values are not allowed in a column.

---

### **147. What Is CHECK?**

It validates values based on a condition.

---

### **148. What Is DEFAULT?**

It automatically assigns a value if no value is provided.

---

### **149. Can a Table Have Multiple UNIQUE Constraints?**

Yes, a table can have multiple unique constraints.

---

### **150. Can a Table Have Multiple Primary Keys?**

A table can have only one primary key constraint, but that primary key can contain multiple columns. This is called a composite primary key.

---

## 151. What Is Composite Primary Key?

A composite primary key uses more than one column to uniquely identify a row.

Example:

```
```sql id="y9wbio" PRIMARY KEY (student_id, course_id)
```

---

## 152. What Is Auto Increment?

Auto increment automatically generates a new numeric value for a primary key.

---

## 153. What Is an Index?

An index improves search performance on columns.

---

## 154. What Is a View?

A view is a virtual table based on a SQL query.

---

## 155. What Happens If We DROP a Table?

The table structure and data are permanently removed.

---

## 156. Does TRUNCATE Remove Table Structure?

No. It removes all rows but keeps table structure.

---

## 157. Is WHERE Allowed with TRUNCATE?

No. `TRUNCATE` removes all rows.

---

## 158. Which Command Is Used to Add a Column?

```
```sql id="mxp6yj"
ALTER TABLE table_name
ADD column_name datatype;
```

---

## 159. Which Command Is Used to Rename a Table?

Depending on database:

```
```sql id="dyh2r1" RENAME TABLE old_name TO new_name;
```

or:

```
```sql id="59qpu1"
ALTER TABLE old_name RENAME TO new_name;
```

---

# PART 17: PRACTICE PROGRAMS

## 160. Basic DDL Practice

1. Create a database named `college_db`.
2. Create a table named `students`.
3. Create a table named `courses`.
4. Add primary key to students.
5. Add primary key to courses.
6. Add NOT NULL constraint to student name.
7. Add UNIQUE constraint to email.
8. Add CHECK constraint to marks.
9. Add DEFAULT value to status.
10. Drop the students table.

---

## 161. Intermediate DDL Practice

1. Create `employees` table.
2. Add `department` column.
3. Modify salary datatype.
4. Rename mobile column to phone\_number.
5. Drop address column.

6. Add unique constraint to email.
  7. Create foreign key between employees and departments.
  8. Create an index on employee name.
  9. Create a view for active employees.
  10. Rename employees table to employee\_details.
- 

## 162. Advanced DDL Practice

Create a training institute database with:

- Students
- Courses
- Trainers
- Batches
- Enrollments
- Payments

Add:

- Primary keys
  - Foreign keys
  - NOT NULL
  - UNIQUE
  - CHECK
  - DEFAULT
  - Indexes
  - Views
- 

## PART 18: PRACTICAL ASSIGNMENTS

### Assignment 1: Student Database

Create a database named:

```
```sql id="mwhrm8" student_db
```

Create a table:

```
```sql id="eyjyyu"
students
```

Columns:

| Column       | Type         | Constraint       |
|--------------|--------------|------------------|
| student_id   | INT          | Primary Key      |
| student_name | VARCHAR(100) | NOT NULL         |
| mobile       | VARCHAR(15)  | UNIQUE           |
| email        | VARCHAR(100) | UNIQUE           |
| course       | VARCHAR(100) | DEFAULT 'Python' |
| marks        | INT          | CHECK 0 to 100   |
| status       | VARCHAR(20)  | DEFAULT 'Active' |

## Assignment 2: Employee Database

Create table:

```
```sql id="cc3lcp" employees
```

Columns:

```
| Column | Type | Constraint |
|---|---|---|
| emp_id | INT | Primary Key |
| emp_name | VARCHAR(100) | NOT NULL |
| department | VARCHAR(50) | NOT NULL |
| salary | DECIMAL(10,2) | CHECK salary > 0 |
| email | VARCHAR(100) | UNIQUE |
| joining_date | DATE | |
| status | VARCHAR(20) | DEFAULT 'Active' |
```

---

## Assignment 3: Product Database

Create table:

```
```sql id="uqs5z2"
products
```

Columns:

| Column       | Type          | Constraint      |
|--------------|---------------|-----------------|
| product_id   | INT           | Primary Key     |
| product_name | VARCHAR(100)  | NOT NULL        |
| price        | DECIMAL(10,2) | CHECK price > 0 |
| stock        | INT           | DEFAULT 0       |
| category     | VARCHAR(50)   |                 |

## Assignment 4: Course Enrollment Database

Create tables:

1. `students`
2. `courses`
3. `enrollments`

Add foreign keys properly.

## Assignment 5: ALTER Practice

Using `students` table:

1. Add `city` column.
2. Add `address` column.
3. Rename `address` to `student_address`.
4. Modify `student_name` size to `VARCHAR(150)`.
5. Drop `city` column.
6. Add unique constraint to mobile.
7. Add check constraint to age.

## PART 19: DDL QUICK REVISION TABLE

| Command         | Syntax                                | Example                                  |
|-----------------|---------------------------------------|------------------------------------------|
| Create Database | <code>CREATE DATABASE db_name;</code> | <code>CREATE DATABASE college_db;</code> |
| Use Database    | <code>USE db_name;</code>             | <code>USE college_db;</code>             |

| Command        | Syntax                                                 | Example                                                     |
|----------------|--------------------------------------------------------|-------------------------------------------------------------|
| Create Table   | <code>CREATE TABLE table_name (...);</code>            | <code>CREATE TABLE students (...);</code>                   |
| Add Column     | <code>ALTER TABLE table ADD column datatype;</code>    | <code>ALTER TABLE students ADD mobile VARCHAR(15);</code>   |
| Modify Column  | <code>ALTER TABLE table MODIFY column datatype;</code> | <code>ALTER TABLE students MODIFY name VARCHAR(150);</code> |
| Drop Column    | <code>ALTER TABLE table DROP COLUMN column;</code>     | <code>ALTER TABLE students DROP COLUMN address;</code>      |
| Drop Table     | <code>DROP TABLE table_name;</code>                    | <code>DROP TABLE students;</code>                           |
| Truncate Table | <code>TRUNCATE TABLE table_name;</code>                | <code>TRUNCATE TABLE students;</code>                       |
| Rename Table   | <code>RENAME TABLE old TO new;</code>                  | <code>RENAME TABLE students TO student_details;</code>      |
| Create Index   | <code>CREATE INDEX idx ON table(column);</code>        | <code>CREATE INDEX idx_email ON students(email);</code>     |
| Create View    | <code>CREATE VIEW view AS SELECT...;</code>            | <code>CREATE VIEW active_students AS SELECT...;</code>      |

## PART 20: 5-DAY DDL TEACHING SCHEDULE

| Day   | Topic                               | Practical                                              |
|-------|-------------------------------------|--------------------------------------------------------|
| Day 1 | SQL Introduction, DDL, Data Types   | Create database and simple tables                      |
| Day 2 | Constraints                         | Create tables with primary key, unique, check, default |
| Day 3 | CREATE Command                      | Create student, employee, product tables               |
| Day 4 | ALTER Command                       | Add, modify, rename, drop columns                      |
| Day 5 | DROP, TRUNCATE, RENAME, Index, View | Complete database design assignment                    |

## PART 21: IMPORTANT POINTS TO REMEMBER

1. DDL means Data Definition Language.
  2. DDL commands work on database structure.
  3. Main DDL commands are CREATE, ALTER, DROP, TRUNCATE, RENAME.
  4. `CREATE DATABASE` creates a new database.
  5. `CREATE TABLE` creates a new table.
  6. `ALTER TABLE` changes table structure.
  7. `DROP TABLE` deletes table permanently.
  8. `TRUNCATE TABLE` removes all records but keeps table structure.
  9. `RENAME` changes object name.
  10. Data types define what kind of data a column stores.
  11. Constraints define rules for data.
  12. Primary key uniquely identifies rows.
  13. Foreign key connects tables.
  14. NOT NULL prevents empty values.
  15. UNIQUE prevents duplicates.
  16. CHECK validates conditions.
  17. DEFAULT automatically assigns value.
  18. Index improves search performance.
  19. View is a virtual table.
  20. Use DROP carefully because it can remove structure and data.
- 

## PART 22: FINAL SUMMARY

DDL commands are used to define and manage database structure.

Important DDL commands:

``text id="hdx05I" CREATE ALTER DROP TRUNCATE RENAME

Important database objects:

``text id="4oa4hx"

Database

Table

View

Index

Constraints

Important constraints:

```text id="qt8vlf" PRIMARY KEY FOREIGN KEY NOT NULL UNIQUE CHECK DEFAULT

Learning flow:

```text id="lze0k6"

SQL Basics

↓

Data Types

↓

Constraints

↓

CREATE

↓

ALTER

↓

DROP

↓

TRUNCATE

↓

RENAME

↓

Index

↓

View

↓

Complete Database Design

The most important skill in DDL is designing clean, meaningful, and scalable table structures before inserting data.

A good database design should have:

- Proper table names
- Proper column names
- Suitable data types
- Primary keys
- Foreign keys
- Required constraints
- Clean relationships
- Future scalability

**DDL is the foundation of database design. Without proper DDL knowledge, we cannot build strong database-based applications.**